Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №3
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Динамические списки»

Выполнили:
студенты группы 25ВВВ2
Ионова А.Н.
Прокофьева А.А.

Приняли:
к.т.н., доцент Юрова О.В.
к.т.н., доцент Деев М.В.

Пенза 2026

Название

Динамические списки

Динамические списки относятся к динамическим структурам и используются, когда размер данных заранее неизвестен. Созданием динамических данных должна заниматься сама программа во время своего исполнения, этим достигается эффективное распределение памяти, но снижается эффективность доступа к элементам.

Динамические структуры данных отличаются от статических двумя основными свойствами:

1) в них нельзя обеспечить хранение в заголовке всей информации о структуре, поэтому каждый элемент должен содержать информацию, логически связывающую его с другими элементами структуры;

2) для них зачастую не удобно использовать единый массив смежных элементов памяти, поэтому необходимо предусматривать ту или иную схему динамического управления памятью.

Для обращения к динамическим данным применяют указатели.

Набор операций над списком будет включать добавление и удаление элементов, поиск элементов списка.

Различают односвязные, двусвязные и циклические списки.

В простейшем случае каждый элемент содержит всего одну ссылку на следующий элемент, такой список называется односвязным.

Задание 1.

Реализовали приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта.

Псевдокод;

```
ФУНКЦИЯ enqueue_priority():
    p = get_struct()                // создаём новый узел
    ЕСЛИ p == NULL: ВЕРНУТЬ

    ЕСЛИ head == NULL:              // список пуст
        head = p
        last = p
        dlina = dlina + 1
        ВЕРНУТЬ

    ЕСЛИ p.prior > head.prior:          // новый элемент самый приоритетный
        p.next = head
        head = p
        dlina = dlina + 1
        ВЕРНУТЬ

    cur = head
    ПОКА cur.next != NULL И cur.next.prior >= p.prior:
        cur = cur.next

    p.next = cur.next               // вставка после cur
    cur.next = p
    ЕСЛИ p.next == NULL:
        last = p
    dlina = dlina + 1


ФУНКЦИЯ dequeue_priority():
    ЕСЛИ head == NULL:
        ВЫВЕСТИ "Очередь пуста"
        ВЕРНУТЬ NULL

    p = head
    head = head.next
    ЕСЛИ head == NULL:
        last = NULL
    p.next = NULL
    dlina = dlina - 1
    ВЕРНУТЬ p


ФУНКЦИЯ priority_menu():
    ЦИКЛ:
        вывести пункты меню (добавить / извлечь / просмотр / найти / удалить / назад)
        choice = ввод
        ВЫБОР choice:
            1: enqueue_priority()
            2: p = dequeue_priority(); ЕСЛИ p != NULL: вывести и free(p)
            3: review()
            4: найти по имени
```

```
          5: del(имя)
          0: выход
    ПОКА choice != 0
```

Результаты выполнения программы:

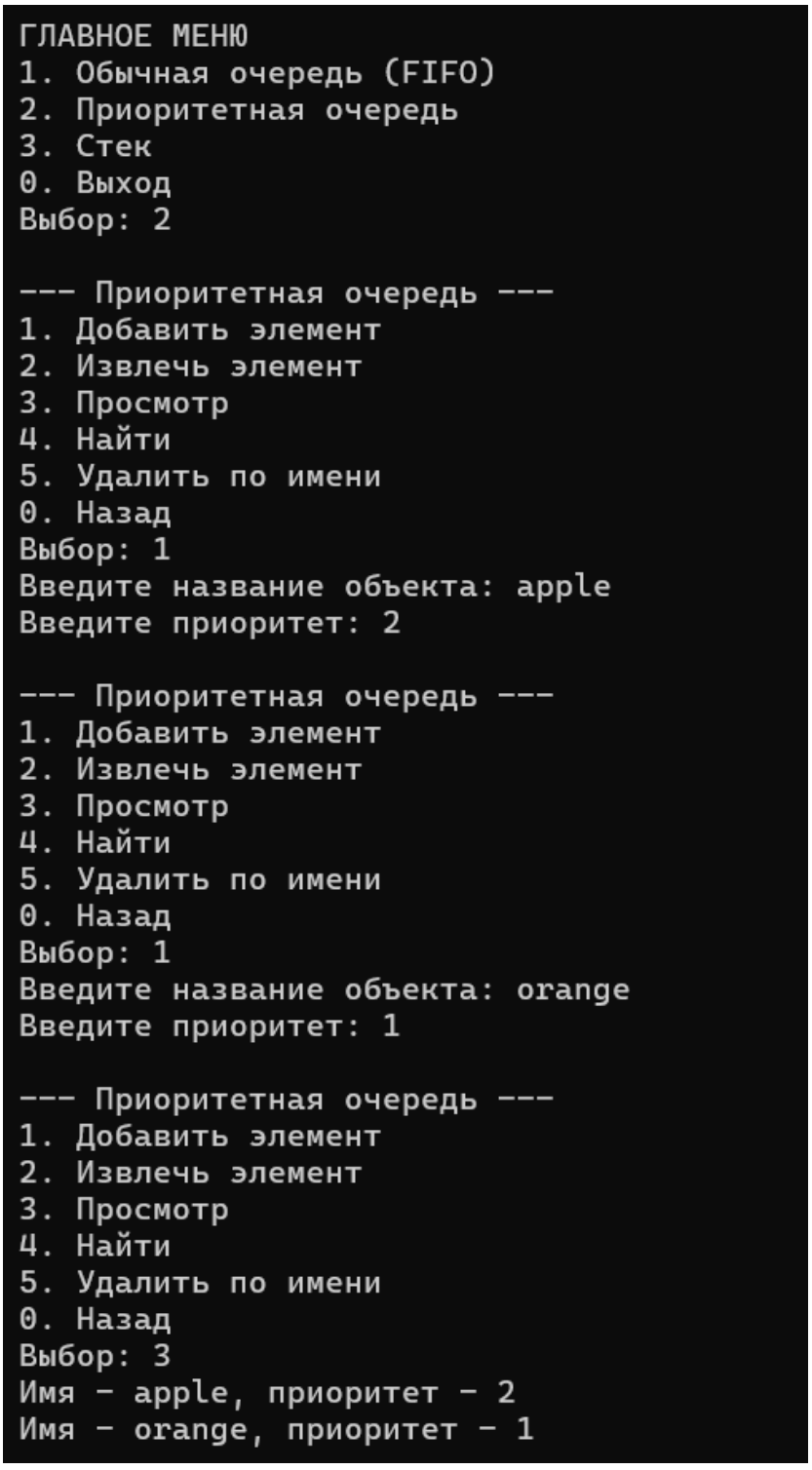


Рисунок 1 – результат выполнения программы «приоритетная очередь»

Задание 2.

На основе приведенного кода реализовали структуру данных *Очередь*.

Псевдокод:

```
ФУНКЦИЯ enqueue_fifo():
    p = get_struct()
    ЕСЛИ p == NULL: ВЕРНУТЬ

    ЕСЛИ head == NULL:              // пустая очередь
        head = p
        last = p
    ИНАЧЕ:                          // добавляем в конец
        last.next = p
        last = p
    dlina = dlina + 1


ФУНКЦИЯ dequeue_fifo():
    ЕСЛИ head == NULL:
        ВЫВЕСТИ "Очередь пуста"
        ВЕРНУТЬ NULL

    p = head                        // берём первый элемент
    head = head.next
    ЕСЛИ head == NULL:
        last = NULL
    p.next = NULL
    dlina = dlina - 1
    ВЕРНУТЬ p


ФУНКЦИЯ fifo_menu():
    ЦИКЛ:
        вывести пункты меню (добавить / извлечь / просмотр / найти
/ удалить / назад)
        choice = ввод
        ВЫБОР choice:
            1: enqueue_fifo()
            2:  p  =  dequeue_fifo();  ЕСЛИ  p  !=  NULL:  вывести  и
free(p)
            3: review()
            4: найти по имени
            5: del(имя)
            0: выход
    ПОКА choice != 0
```

Результаты выполнения программы:

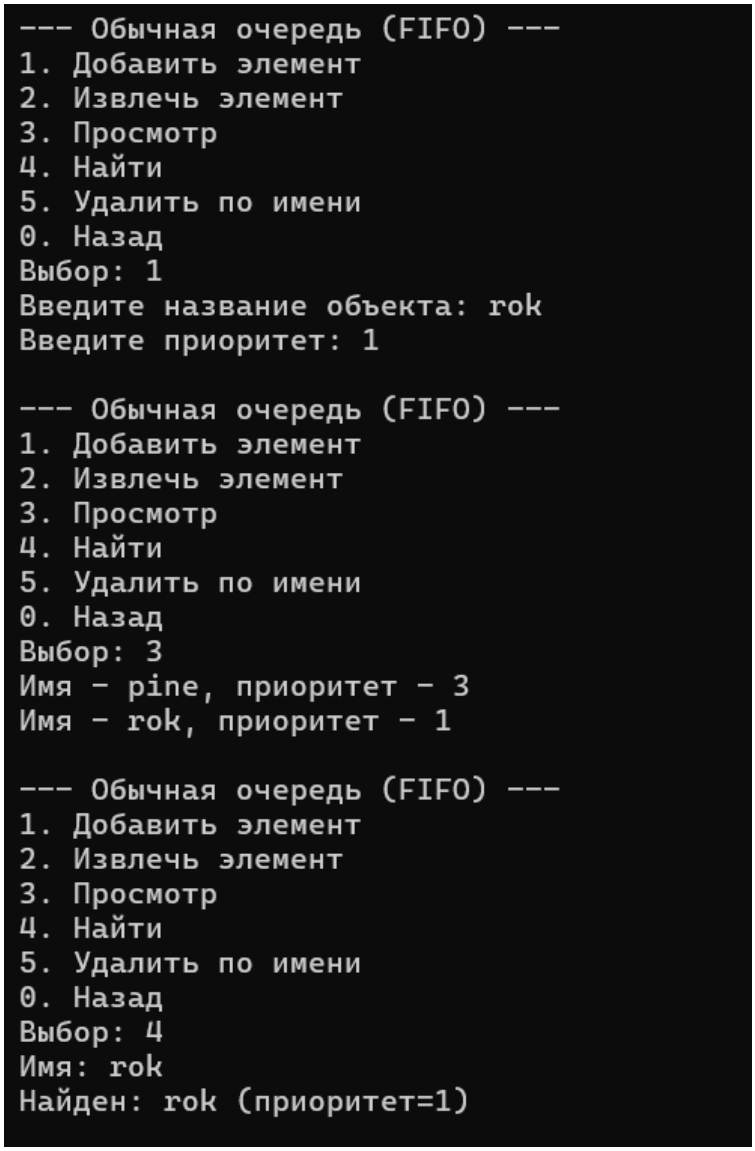


Рисунок 2 – результат выполнения программы «Очередь»

Задание 3.

На основе приведенного кода реализовали структуру данных *Стек.*

Псевдокод:

```
ФУНКЦИЯ push():
    p = get_struct()
    ЕСЛИ p == NULL: ВЕРНУТЬ

    p.next = head                          // новый элемент становится
головой
    head = p
    ЕСЛИ last == NULL:
        last = p
    dlina = dlina + 1


ФУНКЦИЯ pop():
    ЕСЛИ head == NULL:
```

```
            ВЫВЕСТИ "Стек пуст"
            ВЕРНУТЬ NULL

        p = head                          // снимаем верхушку
        head = head.next
        ЕСЛИ head == NULL:
            last = NULL
        p.next = NULL
        dlina = dlina - 1
        ВЕРНУТЬ p


    ФУНКЦИЯ stack_menu():
        ЦИКЛ:
            вывести пункты меню (добавить / извлечь / просмотр / найти
/ удалить / назад)
            choice = ввод
            ВЫБОР choice:
                1: push()
                2: p = pop(); ЕСЛИ p != NULL: вывести и free(p)
                3: review()
                4: найти по имени
                5: del(имя)
                0: выход
        ПОКА choice != 0
```

Результаты выполнения программы:

```
ГЛАВНОЕ МЕНЮ
1. Обычная очередь (FIFO)
2. Приоритетная очередь
3. Стек
0. Выход
Выбор: 3

--- Стек ---
1. Добавить элемент
2. Извлечь элемент
3. Просмотр
4. Найти
5. Удалить по имени
0. Назад
Выбор: 1
Введите название объекта: wer
Введите приоритет: 1

--- Стек ---
1. Добавить элемент
2. Извлечь элемент
3. Просмотр
4. Найти
5. Удалить по имени
0. Назад
Выбор: 3
Имя - wer, приоритет - 1
```

Рисунок 3 – результат выполнения программы «Стек»

Вывод

В ходе лабораторной работы были изучены и практически освоены динамические структуры данных на основе односвязного списка. Реализованы три классические структуры: очередь (FIFO), приоритетная очередь и стек (LIFO), объединённые общим ядром вспомогательных функций.